

CSE 341 - Programming Languages

Homework #1

Sümeyye Zeynep Gürbüz

200104004039

To run the code:

```
~$ clisp Main.lsp
```

~~~~~

### MAIN FUNCTIONS

#### **1. load-file**

*Purpose:*

- Loads the lines of a file into memory as a list.

*How It Works:*

- Opens the specified file and reads each line one by one.
- Uses read-line to retrieve lines from the file.
- Each read line is collected into a list.
- Once reading is complete, the list of lines is returned.

#### **2. read-file**

*Purpose:*

- Recursively reads the next line from the loaded file.

*How It Works:*

- Checks if the current index (curr-ind) is less than the length of the file content.
- If true, it returns the line at the current index and the next index as a list.
- If the end of the file is reached, it returns nil and the current index.

#### **3. write-file**

*Purpose:*

- Writes the entire converted code to the output file.

*How It Works:*

- Opens or creates the specified output file.
- Writes the given line to the file using write-string.
- Adds a newline character after each write operation with write-char.

#### **4. line-type**

*Purpose:*

- Determines the type of a given line.

*How It Works:*

- Extracts the first word of the line.
- Based on the contents of the line, it returns different types like "loop", "if-statement", "return", etc.
- Searches for specific keywords and structures to classify the line appropriately.

#### **5. conversion-foo**

*Purpose:*

Decides which conversion function will be used according to the line type.

*How It Works:*

- Takes the line type returned by line-type and selects the appropriate conversion function.
- If no specific type is matched, it returns 'unknown'.

#### **6. convert**

*Purpose:*

- Converts a line using the specified conversion function and writes the result to a file.

#### *How It Works:*

- Calls the specified conversion function with the line and retrieves the converted line.
- Uses write-file to write the converted line to the output file.

### **7. convert-if**

#### *Purpose:*

- Converts C language if statements to Lisp format.

#### *How It Works:*

- Identifies the condition expression within the parentheses of the if statement.
- Calls replace-operators to convert operators into a suitable Lisp format.
- Formats the result into a valid Lisp if statement.

### **8. convert-operations**

#### *Purpose:*

- Converts basic operations from C to Lisp format.

#### *How It Works:*

- Trims whitespace from the line and splits it into operands and operator.
- Uses format to construct a string representing the operation in Lisp format.

### **9. convert-loops**

#### *Purpose:*

- Converts C for and while loop statements into Lisp loop constructs.

#### *How It Works:*

- For for loops:
  - > It extracts loop parameters and splits them by semicolons to get initialization, condition, and increment/decrement steps.
  - > Determines the loop variable, starting value, and limit.

-> Depending on the comparison operator (>, <), it formats the corresponding Lisp loop construct.

- For while loops:

-> It extracts the condition from the parentheses and converts it using replace-operators.

-> It formats the result as a Lisp loop while statement.

-> Returns an error message if the loop statement is invalid.

## **10. convert-assignments**

*Purpose:*

- Converts variable assignments from C into Lisp let bindings.

*How It Works:*

- It checks for the = operator to identify an assignment.

- Splits the line into variable and value parts.

- Extracts the variable name and the value, trimming any trailing spaces or semicolons.

- Formats the result as a let binding in Lisp.

## **11. convert-functions**

*Purpose:*

- Converts function declarations from C into Lisp defun constructs.

*How It Works:*

- It identifies function names and parameters by searching for the parentheses in the line.

- Extracts the function name and parameters, removing C-specific type keywords (e.g., int, double, char).

- Formats the output as a Lisp defun with the function name and parameter list.

## **12. convert-function-returns**

*Purpose:*

- Converts function return statements from C into Lisp format.

*How It Works:*

- It locates the assignment operator = to separate the function name from its return value.
- Extracts the function name and arguments from the C function call.
- Formats the result as a let binding in Lisp that invokes the function and captures its return value.

### **13. convert-prototypes**

*Purpose:*

- Converts function prototypes from C into Lisp type declarations.

*How It Works:*

- It identifies the function name and parameter list by parsing the line.
- It determines the types of the parameters and the return type, converting C types to Lisp types (e.g., int to integer).
- Formats the result as a declaim statement that specifies the function type.

### **14. convert-calls**

*Purpose:*

- Converts function calls from C into Lisp function call syntax.

*How It Works:*

- It specifically handles printf calls, replacing format specifiers with their Lisp equivalents.
- If the line is a printf call, it formats the arguments for output using format.
- For other function calls, it formats the function name and arguments as a Lisp call.

### **15. convert-returns**

*Purpose:*

- Extracts return values from C return statements.

*How It Works:*

- It locates the return keyword and the following semicolon to identify the return value.
- Trims spaces and returns the extracted value.
- Returns an error message if no return value is found.

## **16. unknown**

*Purpose:*

- Placeholder function for handling unknown operations or lines.

*How It Works:*

- Simply returns the string "unknown".

## **17. convert-parentheses**

*Purpose:*

- Converts parentheses in C to Lisp.

*How It Works:*

- Returns a closing parenthesis as a string.

~~~~~

ADDITIONAL FUNCTIONS

1. replace-operators

Purpose:

- Converts C logical and comparison operators to their Lisp equivalents.
- This function is particularly useful in conditional expressions to ensure Lisp compatibility.

How It Works:

- It takes a condition string and replaces && with and, || with or, == with =, and != with /=.

2. replace-substring

Purpose:

- Replaces all occurrences of a substring within a string.
- Used by replace-operators to handle the actual substring replacement.

How It Works:

- This function iterates through the input str and replaces each occurrence of old with new, writing the modified string to an output.

3. split-sequence

Purpose:

- Splits a string into substrings based on a specified delimiter.
- Useful for parsing C expressions, function parameters, and loop conditions.

How It Works:

- This function divides sequence into substrings separated by delimiter and returns them as a list.

4. remove-tabs-spaces

Purpose:

- Removes leading spaces and tabs from a line.
- This function helps in normalizing code lines by trimming unnecessary

How It Works:

- It finds the first non-space/tab character in the line, removing all preceding whitespace.
- If the line contains only spaces or tabs, it returns an empty string.

5. string-join

Purpose:

- Joins a list of strings with a specified separator.
- Useful for creating formatted Lisp expressions by joining parameters or

conditions.

How It Works:

- It takes a list of strings and joins them with separator.

6. main

Purpose:

- The main function to drive the conversion process.

How It Works:

- Loads the file "Input.c" into content.
- Defines an inner function, process-lines, which iterates through each line:
- Calls line-type to determine the type of line (e.g., loop, assignment).
- Uses conversion-foo to get the appropriate conversion function.
- Calls convert with the cleaned line and the function, executing the line's conversion.
- process-lines is recursively called to process each line until the end of the file.